VIBE CODING CASE STUDY

我如何和 Codex 一起，通过 vibe coding 做出一个应用

这不是一次“AI 帮我写代码”的展示。这是一次“我如何把 AI 变成产品协作者”的复盘。

项目对象

一个把“业务会话归因聚类流程”工具化的小应用。

分享重点

不是产品本身，而是人机协作开发过程和可复用的方法。

关键词：零 coding 经验、AI 协作开发、产品化推进、vibe coding。

一句话结论

这次最大的产出，不是一个应用。而是一套可以复用的人机协作方式。

产品当然做出来了，但真正值得复用的，是这套协作节奏：先收口需求和流程，再分阶段实现，再持续修正口径、状态和反馈。

表层结果

- 做出了本地可用的归因分析工作台。
- 主链路覆盖导入、提取、聚类、分类、处理其他、导出。
- 还有日志、配置、状态反馈和交互修整。

深层结果

- 建立了“人负责判断，Codex 负责展开和执行”的协作边界。
- 形成了可复用的 SOP 和 prompt 组织方式。
- 证明了零 coding 背景也可以把想法推进成产品。

真正被降低的，不只是 coding 门槛，而是把想法变成产品的门槛。

这一页的任务是先把分享重心拉高，不陷入“功能清单”

这个项目不是为了“试试AI”。它解决的是一个真实的高频工作。

面向产品经理自己处理业务会话洞察的场景，核心目标是把过去机械、繁琐、重复的归因聚类流程工具化，从而提效。

业务场景

从大量会话里提炼不购买原因、归因类别、汇总结论。

现实痛点

步骤多、重复度高、机械劳动重，但仍需要业务判断。

产品目标

不是替代判断，而是让流程结构化、结果可复用、类别表可积累。

最适合用 vibe coding 做的，往往不是炫技项目，而是你自己真的反复在做的工作。

第一次没跑通，不是因为 AI 不行。

真正的问题是：流程边界不清，产品结构没定，系统没有闭环。

第一次尝试

- 在和 Codex 开始之前，已经用 opencode 做过一轮。
- 但当时没有把“工具到底该怎么工作”说清楚。
- 结果不是不会写，而是写不成一个能跑通的产品。

第二次重来的变化

- 不是从零开始，而是基于失败成果重写 PRD。
- 先收口流程、对象、状态和导出，再开始 coding。
- 失败经验反而变成了第二次的起点资产。

AI 会放大执行力，但不会替你补上缺失的产品结构。

这一页的价值是建立真实感，也自然引出后面的“结构化比写代码更重要”。

我需要的不是代码生成器。我需要的是一个能持续推进事情的协作者。

这次不是一上来就让 AI 写代码，而是先围绕 PRD、产品边界、对象设计和实现路径展开讨论，再进入 coding。

如果只是“写代码”

- 我依然会卡在产品结构、流程边界和验收判断上。
- 代码写出来了，也不一定能形成一个好用的产品。

这次真正发生的是

- Codex 参与了需求澄清、方案收口、实现推进、debug 修正和文档沉淀。
- 它不是外包开发，而是整个推进过程中的高执行力搭档。

对零 CODING 背景的人来说，最关键的价值

- 先把模糊目标谈清楚，再快速得到可以继续判断的产物。
- 每一轮都能继续推进，而不是卡在“我不知道下一步该做什么”。

当目标还在收敛中时，你需要的不是更快写代码，而是更强的协作推进。

这不是“我把活丢给 AI”。这是判断和执行的清晰分工。

我负责

- 目标定义和问题选择。
- 流程判断、边界取舍、优先级排序。
- 统计口径、体验成立与否、最终验收。

CODEX 负责

- 结构化拆解和工程骨架落地。
- 页面、服务端、数据库、日志、导出等实现工作。
- 定位问题、修复 bug、整理文档、压缩产物。

判断权不外包，执行力被放大。这才是这次协作真正成立的前提。

真正高效的不是“一把梭生成”。而是把项目拆成阶段性闭环。

阶段 1

先把产品结构谈清楚。

任务、批次、对话、类别、当前结果、步骤日志这些对象先被讲清楚。

阶段 2

先搭工作台底座。

任务页、导入、类别表、数据库骨架先落下来，让讨论不再停留在抽象层。

阶段 3

按主链路逐步接通核心能力。

提取、聚类、分类、处理其他、导出，一步一步接，而不是一次性生成。

阶段 4

补日志和状态反馈。

日志页、进度条、按钮状态、保存提示、操作互斥这些体验层开始补齐。

阶段 5

再做口径和分享层收尾。

统一计数口径、补配置入口、完善导出、沉淀 README、PRD 和分享演示稿。

关键不是一次生成全栈，而是每一轮都形成一个可验证的小闭环。

关键决策 1

首版先做“分析工作台”。不做“黑盒自动化”。

如果一开始就做全自动

- 一旦 Prompt、类别定义或“其他”策略不对，会很难排查。
- 错误没有中间结果可看，返工成本会变得很高。

先做工作台的价值

- 每一步都能看输入、输出、状态和日志。
- 方便抽查、修 prompt、修类别表、修重跑策略。

这次为什么证明这个决策是对的

- 上传报错、统计不一致、处理其他无反馈这些问题，都依赖可检查的工作台结构才能快速定位。
- 首版如果做成黑盒，很可能根本不知道错在哪。

在还不稳定的时候，先做可检查，比先做全自动更重要。

先把产品对象讲清楚，很多工程问题就消失了。

先把对象职责讲清楚，再写代码，系统会稳很多。

任务
一套独立归因体系的容器。

批次
一次上传的 CSV 和批次状态。

对话
标准化后的原始会话记录。

类别表
任务级长期资产，可持续迭代。

当前结果
展示、统计、导出的当前生效结果。

步骤日志
用于回溯、debug 和重跑。

很多看起来像工程问题的问题，本质上是产品对象没有定义清楚。

关键收口：最终结果不能靠日志临时拼。

把抽象需求，变成一条清晰的实现链路。

01

导入数据

上传 CSV、字段映射、UTF-8 校验、任务内去重、生成批次。

02

提取不购买原因

要求 LLM 返回结构化 JSON：原因、原文引用、解释、是否存在原因。

03

原因聚类并生成类别建议

先生成建议，再人工确认入表，避免类别表被直接污染。

04

按类别表批量分类

只允许命中一个类别；命不中就进入“其他”。

05

处理其他

只对“其他”局部重算，不整批重跑。

06

汇总与导出

直接从当前结果表联原始对话导出，不靠过程日志拼。

复杂应用不是一次写出来的，而是一条一条链路接出来的。

从需求讨论，到快速搭出工作台底座。

Codex 很强的一个价值，是能把产品讨论快速压缩成一个可运行的工程骨架。

工程初始化

很快落下 Next.js、TypeScript、SQLite、Prisma 的本地单体结构。

页面骨架

任务列表页、任务详情页、类别表、批次导入很快成形。

正反馈

你很快从“抽象讨论”切换到“能点、能看、能判断”的状态。

为什么这一步对零 CODING 背景特别重要

- 以前最容易卡住的是“我根本不知道应该从哪开始”。
- 这次你可以先把问题讲清楚，然后很快获得一个能继续判断的壳子。

主链路不是一把梭生成的。
它是按阶段被接通的。

里程碑 A

接通原因提取

`step_runs`、`llm_call_logs`、当前生效结果表一起落库。

里程碑 B

接通原因聚类

先生成类别建议，再人工确认入正式类别表。

里程碑 C

接通批量分类

按当前类别表逐条归类，命不中统一进入“其他”。

里程碑 D

接通处理其他

只对“其他”局部重算，形成真正闭环。

里程碑 E

补汇总、导出和日志页

从“能跑”升级到“能排查、能复盘”。

AI 协作最强的地方，不是替你跳过过程，而是把过程压缩得足够快。

每一轮都只有一个清晰目标，所以零经验的人也能跟得上推进节奏。

首版不可能全对。
但修正速度真的很高。

这次体验好的地方，不是没有 bug，而是 bug 一旦被明确，就能快速定位、解释、修改和验证。

典型问题

- 上传流程因 Server Action 机制出现 runtime error。
- “7 个其他 vs 5 个其他”暴露出统计口径不一致。
- “处理其他没反应”其实是有逻辑但缺乏反馈。

典型修复

- 上传改成标准 `multipart/form-data` 路由。
- 统一所有批次卡片的计数口径。
- 补运行状态、完成提示、进度条和互斥禁用。

这类修正为什么很有代表性

- 说明 AI 协作真正有杠杆的，不只是“生成”，还有“持续修正”。
- 一个 MVP 是在多轮修正后才从“能跑”变成“能用”的。

有逻辑，不等于有体验。如果用户感知不到状态，功能就像不存在。

这次暴露出来的典型问题

- “处理其他”其实执行成功了，但页面没有反馈，用户会以为按钮坏了。
- 配置保存已经生效了，但没有成功提示，用户不知道有没有改成功。
- 提取、分类、聚类在 mock 下看起来很快，但正式接 LLM 时没有进度条就会显得像“卡住”。

AI 很容易先把“后端逻辑存在”做出来。但产品能不能被信任，常常取决于状态反馈有没有补齐。

这类问题不是技术 bug，而是产品是否成立的问题。

复杂产品里，最容易出错的，往往不是大逻辑，而是口径一致性。

问题表象

- 同一批次在 Step 4 看到 7 个“其他”。
- 但在 Step 5 里又只显示 5 个“其他”。
- 用户自然会怀疑是不是数据出了问题。

根因解释

- 一处用了任务级总计数。
- 一处用了批次级计数。
- 所以不是数据丢了，而是统计口径混了。

AI 能很快实现模块，但如果口径没有被你持续校准，页面就会出现“每块都对、整体不对”的问题。

这类问题尤其需要人来发现，因为它往往不会报错，只会“看起来不对”。

真正有效的 prompt，不是更长，而是更像任务书。

阶段 1

模糊愿望

“帮我做个归因聚类的小工具。”这类说法只够开始，不够稳定推进。

阶段 2

结构化需求

开始明确目标用户、输入输出、页面流程、任务边界和首版范围。

阶段 3

产品级说明

明确对象、状态机、日志、导出字段、步骤职责、不可做项和分阶段交付方式。

这次最有效的 PROMPT 共性

- 目标清楚：到底是做产品，还是只回答问题。
- 边界清楚：先做什么、后做什么、暂时不做什么。
- 格式清楚：要按阶段交付，要有明确字段和结构。

从“帮我做个工具”到“请按三个阶段交付”，中间差的不是字数，是结构。

AI 适合展开。人与人最难替代的是判断。

更适合交给 CODEX

- 工程骨架和数据模型落地。
- 页面、接口、数据库、日志和导出等结构化实现。
- bug 定位、修复、文档整理、内容压缩与重组。

必须由人主导

- 做不做这个产品、首版做什么。
- 流程是否合理、统计口径是否正确。
- 体验是否成立、什么时候该收口、什么时候该返工。

不要把 Codex 当代码外包。要把它当一个高执行力的协作者。

最后做出来的，不是模型壳子。而是一个分步骤、可回溯的工作台。

最终产品不是“上传数据后一键给答案”的黑盒，而是一个更适合首版真实使用的本地分析工作台。

最初设想更像

- 一个自动化很强的归因聚类工具。
- 更偏“上传后跑完整流程”。

最终实际落地

- 一个分步骤、可检查、可修正、可导出的本地工作台。
- 任务管理、CSV 导入、类别表、提取、聚类、分类、处理其他、日志、导出全部接通。

为什么这反而更合理

- 因为这类产品在首版阶段最需要的是稳定、可检查、可追踪，而不是黑盒自动化。
- 后续如果要自动化，也是在这套稳定流程之上继续演进。

如果要复用这次经验，最值得拿走的是一套 SOP。

01

先选一个你自己真的反复在做的流程。

不要先做想象中的 AI 产品，先做你自己最痛的高频工作。

02

不必充分准备好再开始。

但要在推进中尽快把流程、对象和边界讲清楚。

03

先做可检查的工作台，不做黑盒自动化。

中间结果和日志，是你后续所有修正能力的基础。

04

按阶段闭环，不追求一次性生成。

每一轮只追一个清晰验收点，降低复杂度和焦虑感。

05

持续检查状态、口径、反馈和交互。

不要只看“功能逻辑存在”，要看“用户体验是否成立”。

给第一次用 Codex 做项目的人，我最想留三句话。

1

不必充分准备好再开始。先开始，边做边收口。

2

你有没有把结构和流程想清楚，仍然最重要。

3

当前 AI coding 已经把 coding 门槛拉得很低。

不用先成为工程师。先成为一个会定义问题、会收口的人。

这次我最想分享的不是

- “我做出了一个产品”。

而是

- “我如何用 Codex，把一个应用从想法快速推进到可用状态”。

这也是这次案例真正值得传播的部分。

素材补采清单

必须补

- 最终任务工作台截图。
- 最能证明“最后真的做出了一个可用产品”。

建议补

- 第一次失败与重写 PRD 的对比。
- 最能说明“失败经验如何提升第二次起点”。

可选补

- 典型 bug 修复前后对比。
- 最能展示 AI 协作的修正速度。

如果不补素材，这页可以直接跳过，不影响主线表达。

10 分钟分享版删减建议

建议保留

- 1、2、3、4、6、8、13、16、19、20。
- 这 10 页足够讲清背景、失败、分工、关键决策、高光、Prompt 和 SOP。

建议合并

- 11 和 12 合并为“高光时刻”。
- 14 和 15 合并为“翻车与教训”。

建议一句话带过

- 数据库对象、附录类说明、产品功能的细节清单。

10 分钟版本的重点不是完整，而是让听众带走一个清晰方法。

最值得传播的金句

真正被降低的，不只是 coding 门槛，而是把想法变成产品的门槛。

AI 会放大执行力，但不会替你补上缺失的产品结构。

当目标还在收敛中时，你需要的不是更快写代码，而是更强的协作推进。

复杂应用不是被一次性写出来的，而是被一条一条链路接出来的。

先结构化问题，再让 AI 加速执行。

不用先成为工程师。先成为一个会定义问题、会收口的人。

这些句子可以直接拿去做页标题、演讲收束或对外传播文案。